Biweekly Assessment BW04 — Weeks 7–8

Sessions covered: S31–40
Topics: Generators, ksr_utils, modules/venv, OOP (including ABC / interface)

Timebox: 45–60 minutes (homework or Friday of the even week).
Rules: Closed notes unless your trainer allows open notes. No copying.
Coding constraint: Classes and ABC OK. Use print+return for bad amounts — no `try` / `except` (exceptions are Week 9).

A. Multiple Choice (12)

Circle the best option.

1. `yield` makes a function a:
   a) List
   b) Generator
   c) Class
   d) Module
2. Importing your package usually needs:
   a) Only .txt files
   b) MySQL running
   c) React
   d) **init**.py (package marker)
3. `venv` is for:
   a) HTML templates
   b) CSS
   c) Virtual environment / dependencies
   d) Version control
4. `__init__` in a class:
   a) Imports pip
   b) Deletes the object
   c) Runs when instance is created
   d) Is required for print
5. Encapsulation often uses:
   a) Public DB passwords in code
   b) Private-ish attributes + methods
   c) Only global variables
   d) No methods
6. `super()` is used in:
   a) Inheritance to call parent
   b) CSS
   c) Git merge
   d) Generators only
7. Instantiating `class Account(ABC)` that has `@abstractmethod def kind(self)` raises:
   a) SyntaxError
   b) TypeError

c) ValueError
d) KeyError

8. In these notes, a Python “interface” is:
   a) The `interface` keyword
   b) `ABC` with only abstract methods
   c) Any class with `__init__`
   d) A Django model
9. Decorator `@f` above `def g` means:
   a) g = f(g)
   b) f = g(f)
   c) Delete g
   d) Run MySQL
10. `*args` collects extra positional args as a:
    a) tuple
    b) set
    c) str
    d) dict
11. Instance method first parameter is usually:
    a) this
    b) args
    c) self
    d) cls only
12. Polymorphism means:
    a) Only inheritance
    b) One name, different behaviours
    c) One database
    d) No classes

B. Coding (2)

1. Write a generator `read_rows(rows)` that `yield`s each dict from an in-memory list (do not return a full new list).

```
# your code
```

2. Create **SecureBank** `BankAccount` with `__balance`, `deposit`, `withdraw`, `get_balance`. Reject bad amounts with print+return (no `raise` / `try` / `except`).

```
# your code
```

C. Short answer (1)

Why prefer generators for large row streams instead of `return list(...)`?

(Write 3–6 sentences.)